AIMERZ.ai
Aim.Act.Achieve

**NOTES**

# Java Object-Oriented Programming (OOP) - IV

Arrays (1D,2D,3D)

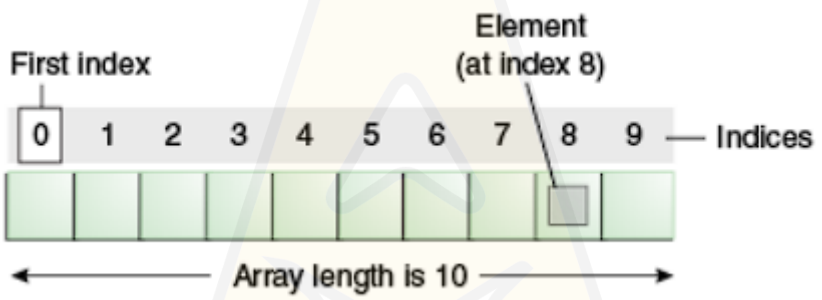
Encapsulation and This keyword

## Arrays in Java

An array is a collection of elements of the same type, stored together in a continuous block of memory. In Java, an array is considered an object that holds multiple elements of a similar data type. These elements are arranged one after another in memory.

A Java array has a fixed size, meaning once it's created, the number of elements it can hold cannot be changed. Arrays are indexed, with the first element at index 0, the second at index 1, and so on.

In short, an array is a structured way to store a set of similar elements in a Java program.




Lets begin with 1 Dimensional Array

### One Dimensional Array (1D)

A list of items can be stored under a single variable name using just one index, and this type of variable is called a one-dimensional array.

**Example**

```
Java
int a[] = new int[5];
```

In this case, the computer allocates five contiguous memory locations for the array.

Example in our Computer

| | | | | |
|---|---|---|---|---|
| a[0] | a[1] | a[2] | a[3] | a[4] |

**Creating an Array**

An array must be declared and initialized in memory before it can be used. The process of creating an array involves three steps:

1. Declare the array.
2. Allocate memory for the array.
3. Assign values to the memory locations.

**Declare the Array**

**Example**

```Java
int[] arr ;
int number[];
float percentage [] ;
int[] count;
float[] average;
```

At the time of declaration we can't specify the size of memory  otherwise we will get a compile time error. The size is nothing but the memory for array we can call also call it array size

**Example**:

```Java
int[] a; //valid

int[10] a; //invalid

int[] a=new int[];//Compilation Error :array dimension missing
```

The allowed data types to specify array size are byte, short, char, int.

By mistake if we are using any other type we will get a compile time error.

**Example**:

**int[] a=new int['a'];  //(valid)**

byte b=10;

**int[] a=new int[b];  //(valid)**

short s=20;

**int[] a=new int[s];  //(valid)**

**int[] a=new int[10l];  //C.E:possible loss of precision//(invalid)**

**int[] a=new int[10.5]; //C.E:possible loss of precision//(invalid)**

The maximum allowed array size in java is maximum value of int size [2147483647].

Example: int[] a1=new int[2147483647]; // (valid)

**int[] a2=new int[2147483648]; // C.E:integer number too large: 2147483648(invalid)**

**Allocate Memory for the Array**

After declaring an array, we must allocate memory for it or we can say allocating size for an array. In Java, we use the **new** operator to create arrays, as shown below:

And, Every array in java is an object hence we can create by using new operator

**Example**

Java

```
ArrayName = new DataType[size / memory];

The array name here is basically the reference variable that points
to the contiguous memory location returned by the new keyword
```

**Example**:

```
Java

number = new int[5];

average = new float[10];
```

These lines allocate memory or size for the array.

We can also combine the declaration and memory allocation in one step, like this:

```
Java

int number[] = new int[5];
```

**Assign values to the memory locations.**

Since arrays are objects in java which comes with index based, and index starts from 0 and and ends with the size which we are giving for our array

In the Java array, each memory location is associated with a number. The number is known as an array index, like if we declared an array whose size is 5, so our java compiler creates an array whose size is 5

**Elements**: elements are nothing but values which we are assigning to our array

Example: new int[5]

Array created whose size 5, means 5 contiguous memory locations created in our computer and ready to take 5 elements

| element1 | elements2 | element3 | element4 | element5 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |

As you can see in above diagram 5 memory locations are allocated for our array and we can see that to address the each memory location java gives us default index values to address each memory location like memory1 or memory2

And these index's start from 0 and ends with the size which we have given

And, For assigning values (**elements**)to each index we have to select the particular index and then assign the value as shown below

```
Java
a[0]=10

a[1]=30

a[2]=45

a[3]=50

a[4]=60
```

In above code we are assigning values to each index in our arry whose size of memory is **5**

**Example 2:**

```
Java
int[] arr = new int[5]; // Array Size is 5
arr[0] = 1; // Values assigned on 0th index
arr[1] = 2; // Values assigned on 1st index
arr[2] = 3; // Values assigned on 2nd index
arr[3] = 4; // Values assigned on 3rd index
arr[4] = 5; // Values assigned on 4th index
```

## Fetching Values from a 1D Array in Java

In a 1D array, each value (called an element) is stored at a specific position, which we call an index. To get or access a value from the array, you use its index.

**Example:**

```
Java

System.out.println(numbers[0]); // Fetches the first element (1)

System.out.println(numbers[3]); // Fetches the fourth element (4)
```

Here, numbers[0] gives you the first element, and numbers[3] gives you the fourth element.

### Fetching Values Using a For Loop

Instead of accessing each element one by one, we can use a **for loop** to easily go through all the elements of the array.

**Example:**

```
Java

for (int i = 0; i < arr.length; i++) {

    System.out.println(numbers[i]);  // Prints each element of the array

}
```

In this loop:

- i is the index, starting from 0.
- numbers[i] gives us each element at index i.
- The loop continues until all elements are printed.

This way, we can easily fetch all values from the array!

### Array initializer

An array initializer is a shorthand way to create and initialise an array in a single line. It uses curly braces {} to specify the array's elements.

Here are the common ways to use array initializers:

```
Java
// Standard array initializer
int[] numbers = {1, 2, 3, 4, 5};
```

### About array initializers:

1. We can't use them when declaring and initialising arrays separately
2. The size is automatically determined by the number of elements
3. All elements must be of the same type (or compatible types)

### Example: Compute Sum and Average of Array Elements

```
Java
class Main {
public static void main(String[] args) {
 int numbers [] = {2, -9, 0, 5, 12, -25, 22, 9, 8, 12};
 int sum = 0;
 Double average;
 // access all elements using for each loop
 // add each element in sum
 for (int number: numbers) {
       sum += number;
 }
 // get the total number of elements
 int arrayLength = numbers.length;
 // calculate the average
 // convert the average from int to double
 average = ((double)sum / (double)arrayLength);
 System.out.println("Sum = " + sum);
 System.out.println("Average = " + average);
}
}
```

**Output**:

```
Java

Sum = 36
Average = 3.6
```

### Iterations in 1D Array using Iteration Statements (Loops)

We can iterate through all the elements in an array using loops

Let's do it by while iterating we will insert elements in any array using **Scanner** class (will be discussed later)

### Example

```
Java
// Inserting elements to an array
Scanner input = new Scanner(System.in);
int[] arr1 = new int[2];

// Iterating in an Array using for loop
for (int i = 0; i < arr1.length; i++) {

// .length, it gives the array length
     System.out.println("Enter Value at Index " + i);
     arr1[i] = input.nextInt();
// Taking input values and assigning to array on each index
}
```

Enter **1, 2** digits from console

Above we can see we are iterating, means traversing in an array and while traversing we are inserting elements also

Now, lets traverse / iterate through an array and fetch all elements which are inserted in an array

### Example

```
Java
for (int i = 0; i < arr1.length; i++) {
     System.out.print(arr1[i]+" ");
}
```

Output **1 2**
Now we can see all the elements present in an array

**2D Array**:

An array of arrays (like a table). In java we can define an array inside another array. This is known as Array of Arrays.

In java multidimensional arrays are implemented as array of arrays approach but not matrix form.

The main advantage of this approach is to improve memory utilization.

Example 1:

```
int[][] a=new int[2][];

a[0]=new int[3];

 a[1]=new int[2];
```

**Two dimensional array declaration:**

Example:

```
int[][] a;
int [][]a;
int a[][];
int[] a[];
int []a[];
```

All are valid.

**Which of the following declarations are valid?**

1. int[] a1, b1;  // a1 and b1 are both 1D arrays (**valid**)
2. int[] a2[], b2 ; // a2 is a 2D array, b2 is a 1D array (**valid**)
3. int[][] a3, b3;  // a3 and b3 are both 2D arrays (**valid**)
4. int[] a, []b;  // Compilation error: <identifier> expected (**invalid**)

Note: If you specify the array dimension before the variable name, this applies only to the first variable in the declaration. You cannot apply the same rule to the second variable in the same line.

Let's see how above 2D array is working, now we can see that in first array whose size 2 is again holding two array whose sizes are also 2

Let's see in a diagram and understand:

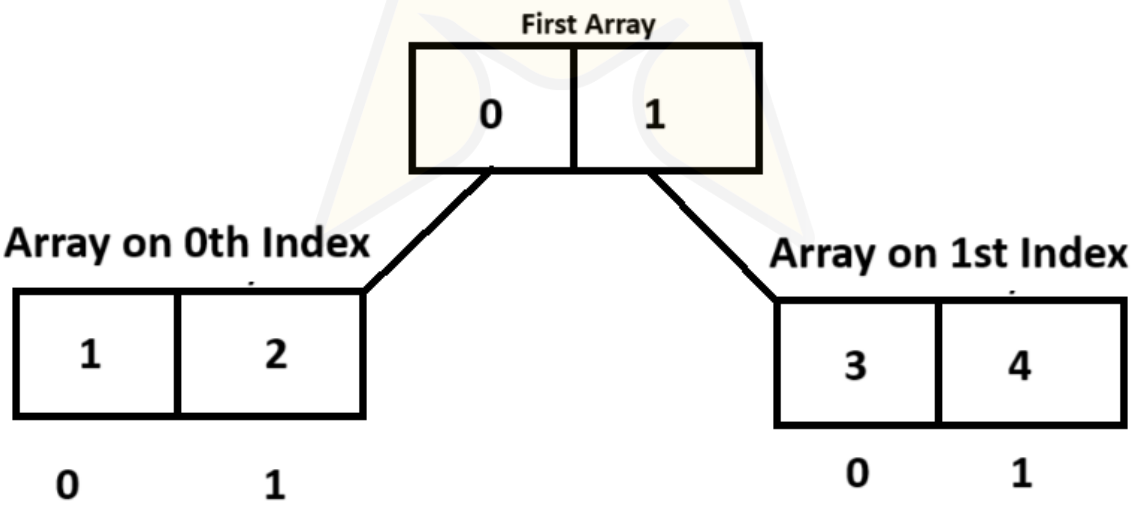
Declaration of the array: int[][] array = new int[2][2];

array[0][0] = 1

array[0][1] = 2

array[1][0] = 3

array[1][1] = 4




In above image we can see that on **0th** index of first array,, there is an array created and **1st** index of first array there is an array created

If we want to access the **0th** element of the array which is present in the **0th** index of first array then we have to address the first array **0th** index and where an array is present and address the **0th** index of inside array

Example : arr[0][0]

Example code for inserting and accessing the elements in array

Example 1

```
Java
int[][] arr = new int[2][2]; // Size of both Arrays is 2
arr[0][0] = 1; // Values assigned on 0th & 0th index
arr[0][1] = 2; // Values assigned on 0th & 1nth index
arr[1][0] = 3; // Values assigned on 1nth & 0th index
arr[1][1] = 4; // Values assigned on 1tth & 1nth index
System.out.println(arr[0][1]+" "+arr[1][1]);
```

Output is **2 4**

**Declaring 2D array with Array Initilizer**

Example

```
Java
int[][] arr = {{1,2},{3,4}};    // Array Initilizer
System.out.println(arr[0][1]+" "+arr[1][1]);
```

Output is **2 4**

**Iteration / Traversal in 2D Array**

Inserting elements into 2D Array while traversing using for loop

Example

```
Java
for (int i = 0; i < arr1.length; i++) {
      for (int j = 0; j < arr1.length; j++) {
            System.out.println("arr["+i+"]"+"["+j+"]");
            arr1[i][j] = input.nextInt();
      }
}
```

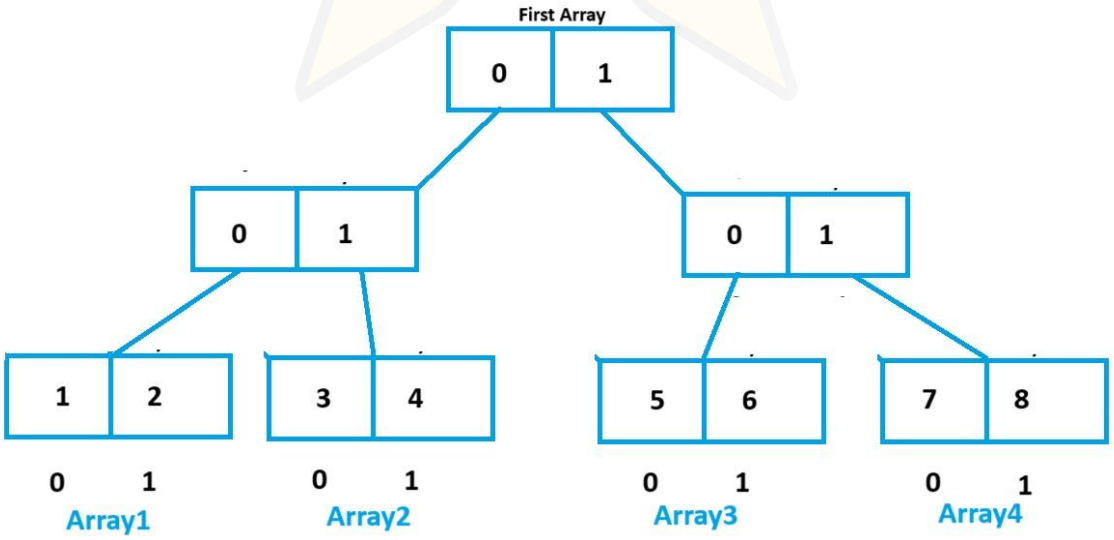
While executing the code it will ask for arr[0][0] and so on, then enter values each time it will ask, and the values will be inserted into the array. And give **1 2 3 4** as input

Now let us fetch the values while traversing in 2D Array
Example

```java
for (int i = 0; i < arr1.length; i++) {
     for (int j = 0; j < arr1.length; j++) {
          System.out.print(arr1[i][j]);
     }
}
```

Output is **1 2 3 4**

**3D Array**: It is a array of 2D arrays
In 3D Arrays we have multiple 2D Arrays




As we can see in above diagram first array consist of an array, and this inside array consist of another array on its **0th** and **1nth** index

And in the same way first **1th** index consist of an array and this inside array consist another on its **0th** and **1nth** indexes

Let's define a 3D array

**Example 1**

```Java
int[][][] arr = new int[2][2][2];
arr[0][0][0] = 1;
arr[0][0][1] = 2;
arr[0][1][0] = 3;
arr[0][1][1] = 4;
arr[1][0][0] = 5;
arr[1][1][1] = 6;
arr[1][1][0] = 7;
arr[1][1][1] = 8;
System.out.println(arr[0][0][0] + " " + arr[1][1][1]);
```

Output is **1 8**

**Now defining 3D Array using Array Initializer**

Example:

```Java
int[][][] cube = {{{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}};
System.out.println(cube[1][1][0]);  // Outputs 7
```

**iteration / Traversal in 3D Array**

Lets insert elements while traversing in 3D Array

**Example**

```Java
// Scanner Object created
Scanner input = new Scanner(System.in);
// 3D Array Created and Size is 2
```

```
int[][][] arr1 = new int[2][2][2];
// Iterating in 3D Array
for (int i = 0; i < arr1.length; i++) {
     for (int j = 0; j < arr1.length; j++) {
          for (int k = 0; k < arr1.length; k++) {
System.out.println("arr[" + i + "]" + "[" + j + "]" + "[" + k + "]");
          arr1[i][j][k] = input.nextInt();
          }
      }
}
```

While this code executes it will iterating / traversing and will ask for input on each iteration give **1 2 3 4 5 6 7 8**

Now let's write code printing all elements in 3D array

**Example**

```
Java
for (int i = 0; i < arr1.length; i++) {
     for (int j = 0; j < arr1.length; j++) {
          for (int k = 0; k < arr1.length; k++) {
               System.out.print(arr1[i][j][k]+" ");
                }
            }
        }
```

Output **1 2 3 4 5 6 7 8**

**Anonymous Arrays:**

- Sometimes we can create an array without a name, and such arrays are called **anonymous arrays**.
- The primary purpose of anonymous arrays is for "instant use."
- We can create an anonymous array like this:

```
Java
new int[]{10, 20, 30, 40}; // (valid)
new int[][]{{10, 20}, {30, 40}}; // (valid)
```

When creating an anonymous array, we cannot specify the size, as this would cause a compilation error.

**Example**:

```Java
new int[3]{10, 20, 30, 40}; // Compilation error: ';' expected (invalid)
new int[]{10, 20, 30, 40};  // (valid)
```

If needed, we can assign a name to an anonymous array, making it no longer anonymous:

**Example**:

```Java
int[] a = new int[]{10, 20, 30, 40}; // (valid)
```

**Example**:

```Java
public class AnonymousArrayExample {
    public static void main(String[] args) {
        // Using an anonymous array directly
        sum(new int[]{10, 20, 30, 40});
       // passing an anonymous array to a method
    }

    public static void sum(int[] numbers) {
        int total = 0;
        for (int num : numbers) {
            total += num;
        }
        System.out.println("Sum: " + total);
    }
}

// Output: Sum: 100
```

### Passing Arrays to Methods:

Just like we can pass primitive values to methods, we can also pass arrays. For example, the following method prints all the elements in an int array:

```
Java
public static void printArray(int[] array) {
    for (int i = 0; i < array.length; i++) {
        System.out.print(array[i] + " ");
    }
}
```

We can call this method by passing an array. For instance, the following statement will display 3, 1, 2, 6, 4, 2:

```
Java
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

### Returning an Array from a Method:

A method can also return an array. For example, the method below returns a reversed version of an array:

```
Java
public static int[] reverse(int[] list) {
    int[] result = new int[list.length];
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--) {
        result[j] = list[i];
    }
    return result;
}
```

This method takes an array, reverses its elements, and returns the new array.

## ArrayIndexOutOfBounds Exception

**What is it?**
An ArrayIndexOutOfBoundsException is an error that occurs when you try to access an index of an array that doesn't exist.

**Why does it happen?**
In Java, array indexes start from **0**. For example, if you have an array with 5 elements, valid indexes are **0** to **4**. Trying to access an index like **5** or **-1** will cause this exception.

**Example**:
If you have an array defined as follows:

```java
int[] numbers = {10, 20, 30, 40, 50};
```

And we try to access:

```java
System.out.println(numbers[5]); // This will cause ArrayIndexOutOfBoundsException
```

In this case, you're trying to access the 6th element, which doesn't exist.

**How to avoid it:**

Always check that your index is within the valid range:

```java
if (index >= 0 && index < numbers.length) {
```

```
    System.out.println(numbers[index]);
}
```

Use loops carefully, making sure they don't go out of bounds.

An **ArrayIndexOutOfBoundsException** is a common mistake when working with arrays. Understanding array indexing and carefully managing your index values can help you avoid this error.

## Enhanced For Loop in Java

The enhanced for loop, also known as the for-each loop, is a simpler way to iterate through arrays and collections in Java. Introduced in JDK 5, this loop makes it easier to read and write code by eliminating the need for explicit index management.

### Syntax of the Enhanced For Loop

The syntax for the enhanced for loop is straightforward:

```Java
for(dataType variable : arrayOrCollection) {
    // code to be executed
}
```

**Explanation:**

**dataType**: The type of elements in the array or collection.

**variable**: A temporary variable that holds each element during iteration.

**arrayOrCollection**: The array or collection you want to iterate over.

**Real-World Analogy**

Think of the enhanced for loop like a conveyor belt in a factory. Each item on the belt (the elements of the array or collection) passes by, and you can take each one without having to worry about where it came from or how many items are left.

### Basic Example of Iterating in an Array

```java
class ForEachExample {
    public static void main(String[] args) {
        int[] evenNumbers = {2, 4, 6, 8};

        for (int number : evenNumbers) {
            System.out.println(number);
        }
    }
}
```

Output

```
2
4
6
8
```

In this example, for each iteration, number takes on the value of each element in the evenNumbers array.

```java
class SumExample {
    public static void main(String[] args) {
        int[] numbers = {3, 4, 5, -5, 0, 12};
        int sum = 0;

        for (int number : numbers) {
            sum += number; // Add each number to sum
        }

        System.out.println("Sum = " + sum);
    }
```

```
}
```

**Output**

```java
Sum = 19
```

### Advantages of Enhanced For Loop

Simplicity: The enhanced for loop reduces boilerplate code and makes it easier to read.

Less Error-Prone: It eliminates common errors associated with traditional loops, such as off-by-one errors when accessing array indices.

Readability: The intent is clearer; it's obvious that you're iterating through all elements.

### Limitations of Enhanced For Loop

No Index Access: You cannot access the index of the current element directly. This means you can't modify elements at specific positions.

Sequential Only: The enhanced for loop only iterates in a forward direction; you cannot iterate backward or skip elements.

Not Suitable for All Use Cases: If you need to manipulate elements based on their index or need to iterate in reverse order, a traditional for loop is more appropriate.

### Comparison with Traditional For Loop

| Feature | Enhanced For Loop | Traditional For Loop |
|---|---|---|
| Syntax | for(dataType var : array) | for(int i = 0; i < length; i++) |
| Index Access | No | Yes |
| Direction | Forward only | Forward and backward |
| Use Case | Simple iterations | Complex iterations |

### Iterating in 1D Array using for each loop

```Java
int[] arr = {10,20,30,401};
      for (int i : arr) {
        System.out.println(i);
      }
```

### Output

```Java
10
20
30
401
```

### Iterating in 2D Array using for each loop

```Java
int[][] arr = {{10,20,30},{40,50,60}};

        for (int[] is : arr) {
            for (int val : is) {
```

```
                System.out.println(val);
            }
        }
```

**Output**

```
Java
10
20
30
40
50
60
11
```

**Iterating in 3D Array using for each loop**

```
Java
int[][][] arr = {{{5,6,7},{8,9,10},{11,12,13}}};

        for (int[][] first : arr) {
            for (int[] second : first) {
                for (int third : second) {
                    System.out.println(third);
                }
            }
        }
```

Output

```
Java
5
6
7
8
9
10
11
12
13
```

## Encapsulation & this Keyword

Encapsulation is one of the four core principles of Object-Oriented Programming (OOP). It involves bundling data (variables) and methods (functions) that work on that data into a single unit, or class, while limiting access to certain parts of the object. This approach safeguards the object's integrity and helps maintain its internal state.

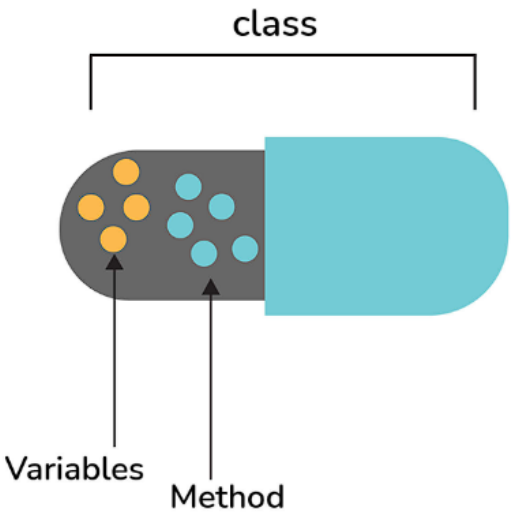
In programming, encapsulation means hiding the complex details inside a piece of code (like the data or variables inside an object) and only allowing access to the important parts. To do this, we keep the data private, so it's not directly accessible from the outside. Instead, we use special methods called getters and setters to safely read or change the data. This way, we protect the internal workings and make sure only safe, necessary parts are used.

**Encapsulation** is a concept in object-oriented programming (OOP) that is about **hiding data** and providing safe access to it.

```
class
{

  data members (properties)
          +
   methods (behavior)

}
```




Think of a bank account as an example of encapsulation. A bank account has data (account number, account holder's name, balance) and methods (deposit, withdraw, check balance). The data is hidden from the outside world, and the methods are the only way to interact with the data. You can't directly access the balance or modify it without using the provided methods.

**Real-world analogy: DepositBox**
Imagine a safe deposit box at a bank. You can't just walk into the bank and access the contents of the safe deposit box. You need to use the key (method) provided by the bank to open the box and access the contents (data). The bank controls who can access the box and what actions can be performed on the contents.

**Real-World Analogy : Capsule**
Imagine a **capsule medicine**. The capsule's shell **encapsulates** (surrounds) the medicine, keeping it protected inside. You can't access the medicine directly; you can only use it in a controlled way (by swallowing the capsule).

Similarly, in Java:
The **private data** in a class is like the medicine.
The **public methods** (getter and setter methods) are like the capsule's shell that controls access to the medicine.

### Why is Encapsulation Important?

Encapsulation provides:

**Data Protection**: By keeping data private, it prevents unauthorized changes or incorrect values from being set.

**Control Over Data**: Only specific methods can access or modify the data, allowing validation or checks to be applied.

**Easy Maintenance**: If the data changes, you only need to modify the code inside the class.

### How to Achieve Encapsulation in Java?

In Java, we use:

- **Private access modifier** for variables to make them inaccessible outside the class.
- **Public getter and setter methods** to access and modify these variables.

### Example 1: Small Example – Encapsulating a Bank Account Balance

```Java
class BankAccount {
    // Private data member
    private double balance;

    // Constructor to set initial balance
    public BankAccount(double initialBalance) {
        balance = initialBalance;
    }

    // Public method to get balance
    public double getBalance() {
        return balance;
    }

    // Public method to set balance with validation
    public void deposit(double amount) {
```

```
        if (amount > 0) {
            balance += amount;
        } else {
            System.out.println("Deposit amount should be positive.");
        }
    }
}
```

### Explanation

Here, the balance is private, meaning it's hidden from outside the class.

The methods getBalance and deposit provide safe access to balance. For example, deposit checks if the amount is positive before adding it to the balance.

### Real-World Use Case of Encapsulation

Think about an **employee management system** in a company. Here, we need to protect sensitive information like salary details.

### Example 2: Employee Details Encapsulation

```Java
class Employee {
    // Private data members
    private String name;
    private double salary;

    // Constructor
    public Employee(String empName, double empSalary) {
        name = empName;
        salary = empSalary;
    }

    // Getter for name
    public String getName() {
        return name;
    }

    // Setter for name
    public void setName(String empName) {
        name = empName;
```

```
    }

    // Getter for salary
    public double getSalary() {
        return salary;
    }

    // Setter for salary with validation
    public void setSalary(double empSalary) {
        if (empSalary > 0) {
            salary = empSalary;
        } else {
            System.out.println("Salary must be positive.");
        }
    }
}
```

**Explanation**

The salary and name fields are private, so only the methods in the Employee class can access or modify them.

The setter setSalary checks if the salary is positive, controlling the data safely.

**Real-time usage:**

Encapsulation is used extensively in real-world applications, such as:

**Banking systems**: As shown in the example above, bank accounts are encapsulated to ensure secure and controlled access to account information.

**E-commerce platforms:** User data, such as passwords and credit card information, is encapsulated to prevent unauthorized access.

Social media platforms: User profiles and data are encapsulated to control who can view and interact with the information.

### What is 'this' Keyword in Java?

The, **this** keyword in Java refers to the current instance of the class. It's often used when there's a naming conflict between instance variables and method parameters or to invoke other constructors.

The, this keyword is a reference to the current object of the class. It's used to access the members (methods and variables) of the current object.

Think of this as a pointer to the current object. When you use this inside a method, it refers to the object that the method is being called on.

### Real-world analogy:

**" Imagine you're at a party, and you want to introduce yourself to someone. You would say "Hi, I'm [Your Name]". The word "I" refers to yourself, the person speaking. Similarly, in programming, this refers to the current object, the one that the method is being called on. "**

## Using this Keyword to Avoid Naming Conflict / Shadow Problem

When a constructor or method parameter has the same name as an instance variable, we use **this** to differentiate them.

### Example without 'this' keyword

```Java
class Student {
    private String name;

    // Constructor with parameter name same as instance variable
    public Student(String name) {
       name = name;  // Here the name conflict arise
    }
    public String getName() {
        return name;
    }
}
```

In this example we tried to bind the local variable 'name' with instance variable 'name' but due to same variable names the name conflict is arising at the time of compilation

**Let's see how**

```java
public static void main(String[] args) {
        Student std = new Student("Aimerz");

        System.out.println("The name is: "+std.name);
}
```

**Output**

```java
The name is: null
```

See above after passing a String argument to the constructor but still our instance variable 'name' have not been defined with "Aimerz" and we are getting null has output

So this is naming conflict problem which we are seeing, so how to solve using **this** keyword

**Example with 'this' keyword**

```java
class Student {
    private String name;

    // Constructor with parameter name same as instance variable
    public Student(String name) {
        this.name = name; // 'this.name' refers to the instance variable,
'name' to the parameter
    }

    public String getName() {
```

```
        return name;
    }
}
```

In this example:

this.name refers to the instance variable name of the class.

Without this, Java wouldn't know if name refers to the instance variable or the parameter.

### Bigger Example – Order Management System

Consider an **order management system** in an online store where each order has details like item, quantity, and a discount.

```Java
class Order {
    private String item;
    private int quantity;
    private double discount;

    // Constructor with parameters having the same name as instance
variables
    public Order(String item, int quantity, double discount) {
        this.item = item;        // Using 'this' to refer to instance
variables
        this.quantity = quantity;
        this.discount = discount;
    }

    // Method to display order details
    public void displayOrder() {
        System.out.println("Item: " + item);
        System.out.println("Quantity: " + quantity);
        System.out.println("Discount: " + discount + "%");
    }
}

public class OrderManagementSystem {
```

```
    public static void main(String[] args) {
        Order order = new Order("Laptop", 2, 10.0); // Creates an order with
a discount
        order.displayOrder();
    }
}
```

**Explanation**

In the Order class:

this.item, this.quantity, and this.discount refer to the instance variables.

Without this, it would be unclear which item, quantity, and discount are instance variables and which are parameters.

**Output**

```
Java
Item: Laptop
Quantity: 2
Discount: 10.0%
```

**Real-time usage:**

The this keyword is used extensively in real-world applications, such as:

**GUI programming:** When creating graphical user interfaces, this is used to refer to the current window or component.

Game development: In game development, this is used to refer to the current game object or character.

**Web development:** In web development, this is used to refer to the current HTML element or JavaScript object.

# THANK YOU

AIMERZ.ai

Aim.Act.Achieve

**Stay updated with us!**

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan